



Graph-Augmented Retrieval-Augmented Generation for Vietnamese Labor Law Question Answering

Thesis Report

Supervisor: Dr. Tran Hoang Tung

Student:

Tran Hien Chuong 22BA13056

Academic Year 2025–2026

Abstract

TODO: Write a concise 200–300 word abstract after chapter drafting. Mention the scoped Vietnamese labor-law corpus, hierarchy-aware chunking, Qdrant, Neo4j, graph-augmented retrieval, extractive citation-grounded answering, deterministic rule-based validation, and the constructed 69-query benchmark.

Acknowledgements

TODO: Add acknowledgements before submission.

Contents

Abstract	i
Acknowledgements	ii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Research Objectives	1
1.4 Scope of the Project	1
1.5 Contributions	1
1.6 Thesis Structure	2
2 Background and Related Work	3
2.1 Large Language Models	3
2.2 Retrieval-Augmented Generation	3
2.3 Legal Question Answering and Domain Challenges	3
2.4 Knowledge Graphs and Graph-Augmented RAG	4
2.5 Evaluation Methods for RAG Systems	4
3 System Design and Methodology	5
3.1 Overall System Architecture	5
3.2 Legal Corpus and Data Sources	7
3.3 Legal Document Preprocessing and Enrichment	9
3.4 Legal Cross-Reference Extraction and Graph Design	13
3.5 Vector-based and Hybrid Retrieval	17
3.6 Query Routing and Graph Expansion Policy	20
3.7 Grounded Answer Generation with Legal Citations	23
4 Implementation	28
4.1 Technology Stack	28
4.2 Data Processing Implementation	28
4.3 Vector Index and Hybrid Retrieval Implementation	29
4.4 Neo4j Graph Implementation	29
4.5 Graph-Augmented Retriever Implementation	29
4.6 Grounded Answer Generator and Validation	29
4.7 Evaluation Scripts and Reproducibility	30
4.8 Example Use Cases	30

5	Evaluation and Results	31
5.1	Evaluation Setup and Benchmark Design	31
5.2	Retrieval Evaluation Methodology	32
5.3	Answer and Citation Evaluation Methodology	32
5.4	Retrieval Results	32
5.5	End-to-End Results	33
5.6	Case Studies	34
5.7	Error Analysis and Limitations	34
6	Discussion	35
6.1	Key Findings	35
6.2	Impact of Hierarchical Chunking	35
6.3	Benefits of Graph-Augmented Retrieval	35
6.4	Role of Normative Hierarchy in Legal QA	35
6.5	Citation Grounding and Hallucination Control	35
6.6	Practical Applications	35
6.7	Limitations	36
7	Conclusion and Future Work	37
7.1	Conclusion	37
7.2	Future Work	37
	References	38
A	List of Legal Documents	39
B	Chunk Schema	40
C	Graph Schema	41
D	Benchmark Samples	42
E	Evaluation Metrics	43
F	Reproducibility Commands	44
G	Example Retrieved Contexts and Answers	45

List of Figures

2.1	Standard retrieval-augmented generation pipeline.	3
3.1	Overall architecture of the proposed graph-augmented RAG system. . . .	6
3.2	Legal data processing pipeline with hierarchy-aware chunking and meta- data preservation.	10
3.3	Conceptual schema of the Neo4j legal knowledge graph.	14
3.4	Graph-augmented retrieval flow combining hybrid seed retrieval and legal graph expansion.	24
3.5	Answer generation and deterministic citation validation flow.	27

List of Tables

3.1	Summary of the scoped Vietnamese labor law corpus and document meta-data.	7
3.2	Legal chunking and data integrity validation results.	12
3.3	Summary of loaded node and edge counts in the Neo4j legal knowledge graph.	15
4.1	Technology stack.	28
5.1	Corpus and index statistics.	31
5.2	Retrieval modes comparison table.	32
5.3	Category-level result table.	33
5.4	End-to-end evaluation summary table.	33
6.1	Limitations table.	36
A.1	Appendix document list placeholder.	39

Chapter 1

Introduction

1.1 Background

TODO: Introduce Vietnamese labor law as a structured, citation-heavy information access domain. Explain why users need answers that preserve article, clause, point, document, and normative-rank context rather than only keyword matches.

1.2 Problem Statement

TODO: State the concrete problem: retrieval-only legal QA can miss cross-referenced provisions, implementing guidance, appendices, and procedure rules, which can lead to incomplete or weakly grounded answers.

1.3 Research Objectives

TODO: List measurable objectives: build a Vietnamese labor-law Graph-Augmented RAG pipeline, construct a hierarchy-aware corpus and graph, retrieve citation-grounded evidence, generate extractive answers with legal citations, and evaluate on the constructed 69-query benchmark.

1.4 Scope of the Project

TODO: Define the project scope around the six indexed Vietnamese labor-law and labor-procedure documents. Clarify that the system is for information assistance within the indexed corpus and is not a substitute for professional legal advice.

1.5 Contributions

TODO: Summarize contributions: cleaned legal corpus, hierarchy-aware legal chunking, metadata enrichment, cross-reference extraction, Qdrant vector index, Neo4j legal knowledge graph, graph-augmented retrieval, citation-grounded extractive answer generation, and deterministic validation artifacts.

1.6 Thesis Structure

TODO: Briefly describe Chapters 2–7 after drafting: background and related work, methodology, implementation, evaluation, discussion, and conclusion with future work.

Chapter 2

Background and Related Work

2.1 Large Language Models

TODO: Explain transformer-based language models, their use in question answering, and their limitations for legal domains, including hallucination, context limits, and the need for source grounding. Add verified citations.

2.2 Retrieval-Augmented Generation

TODO: Move and expand the old RAG background here. Explain indexing, embeddings, retrieval, context assembly, and grounded generation. Cite foundational RAG work such as [1] and add any newer verified RAG references.

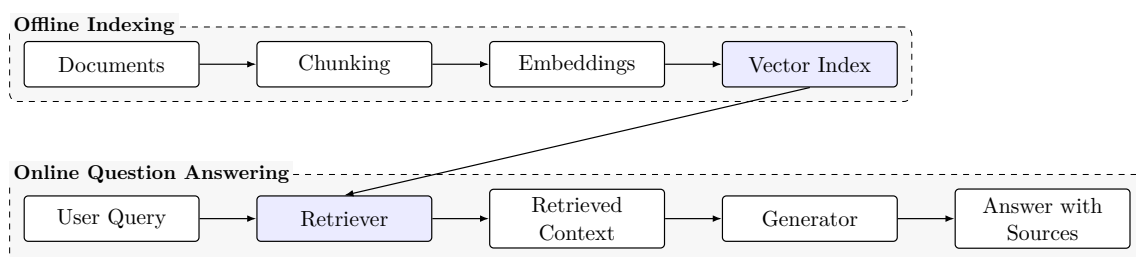


Figure 2.1: Standard retrieval-augmented generation pipeline.

2.3 Legal Question Answering and Domain Challenges

TODO: Legal QA requirements. Review legal QA requirements: jurisdiction specificity, temporal validity, citation traceability, cautious wording, and the risk of unsupported legal conclusions. Add legal QA references using [4].

TODO: Legal document structure and citation. Explain how Vietnamese legal documents are organized into documents, chapters, sections, articles, clauses, points, and appendices. Discuss why legal citations must preserve this hierarchy. Cite verified Vietnamese legal document sources using [2].

2.4 Knowledge Graphs and Graph-Augmented RAG

TODO: Knowledge graphs for legal retrieval. Introduce knowledge graphs for representing legal hierarchy, references, topics, actors, issue types, and normative relationships. Explain how graph structure complements embedding similarity. Add verified KG and legal KG references.

TODO: Graph-augmented retrieval. Move the old graph-augmented retrieval background here. Compare graph expansion with vector-only and hybrid retrieval, and cite GraphRAG or graph-augmented retrieval work using [3].

2.5 Evaluation Methods for RAG Systems

TODO: Review retrieval metrics, answer validation, and citation grounding. Distinguish human expert review, LLM-as-Judge, and deterministic rule-based validation; later chapters use deterministic rule-based validation only.

Chapter 3

System Design and Methodology

3.1 Overall System Architecture

Figure 3.1 presents the proposed Graph-Augmented Retrieval-Augmented Generation (GraphRAG) architecture tailored for high-precision Vietnamese labor-law question answering. To eliminate stochastic volatility and enforce structural compliance within a strict regulatory domain, the system is designed as an integrated framework divided into two primary operating pipelines: an **Offline Ingestion Pipeline** that transforms unstructured statutory texts into deeply parented, relational knowledge layers, and an **Online Query-Routed Retrieval and Answering Pipeline** that drives adaptive context expansion and deterministic generation guardrails.

The **Offline Ingestion Pipeline** establishes the foundational data layer required for dense vector matching and topological graph traversals. The pipeline ingests raw Vietnamese legal documents and routes them through a rigorous text-cleaning phase to eliminate OCR artifacts and normalize formatting metadata. Rather than using standard flat, fixed-size windowing strategies that cause semantic fragmentation, the cleaned stream is parsed via a syntax-aware parser utilizing deterministic regular expression grammars. This parser executes *Hierarchical Chunking*, isolating legal units at the point (*Điểm*) or clause (*Khoản*) level while preserving structural dependencies. These granular fragments are then wrapped within a specialized *Semantic Preservation Layer*, which leverages metadata inheritance to inject absolute statutory addresses and document ranks directly into the chunk payloads. Finally, the enriched units are indexed simultaneously across two complementary structures within the **Knowledge Stores**: a hybrid vector index in Qdrant for semantic similarity searches, and a deterministic document-structure legal knowledge graph in Neo4j to model cross-references and statutory hierarchies.

The **Online Query-Routed Retrieval and Answering Pipeline** governs query execution at runtime, isolating semantic intent resolution from evidence retrieval. When a conversational question is submitted, it is ingested by the *Query Router*, which combines constrained language model inference with rule-based dictionaries to extract discrete entities, topics, issues, and targeted article numerals, consolidating them into a unified **QueryIntent** control object. This control object dynamically configures the downstream execution engine, activating specialized filtering constraints and routing reference-boost behaviors directly through Qdrant’s sparse vector prefetch channel. The retrieval layer executes a *Multi-Query Multi-Modal Parallel Prefetching* strategy, pooling dense and sparse candidates before merging them via Reciprocal Rank Fusion (RRF).

To safeguard against embedding dilution and capture complex document-guidance

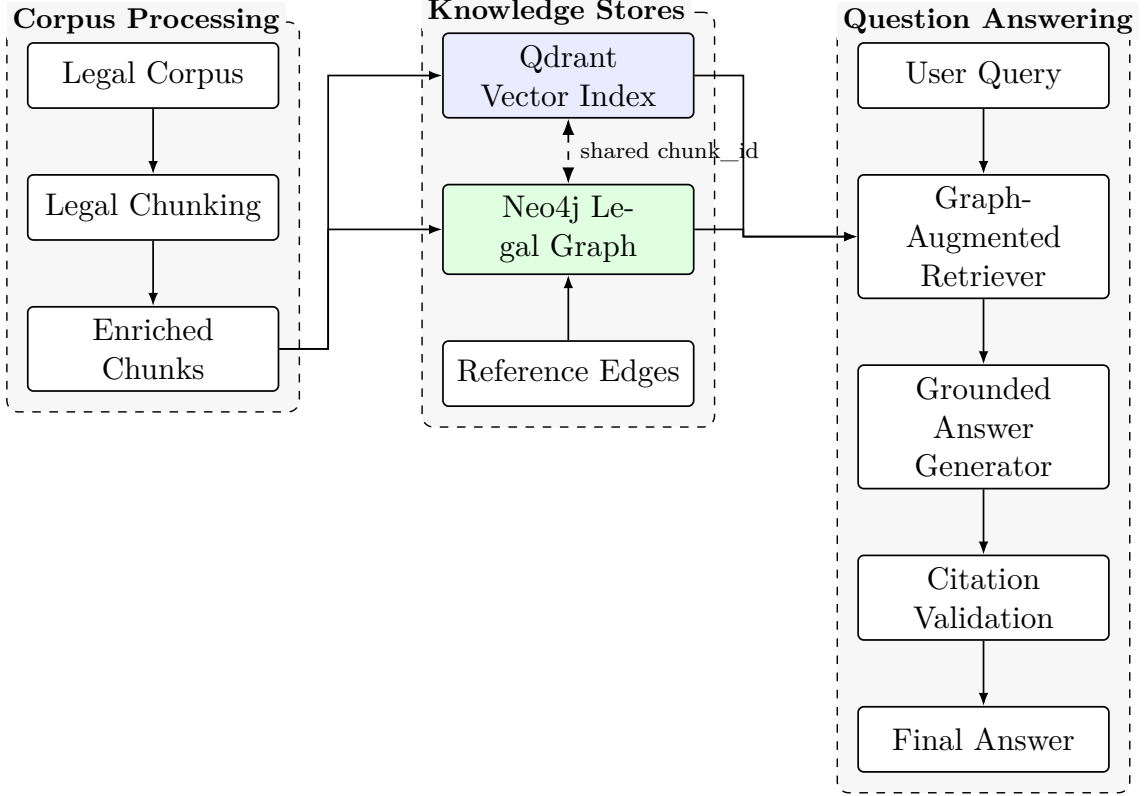


Figure 3.1: Overall architecture of the proposed graph-augmented RAG system.

links, the pooled baseline results are passed to the *Graph-Augmented Retriever*. The retriever treats the vector hits as initial seed anchors and leverages the Neo4j schema to execute relational graph expansions, pulling in geographically distant but legally relevant circular guidance or implementing decree clauses. This expansion is governed by an *Adaptive Context Window Strategy* that dynamically expands chunk limits only when complex multi-hop queries or mathematical lookup procedures are detected. Concurrently, a *Forced Reference Promotion Policy* and a *Deterministic Coordinate Fallback Policy* run in parallel to guarantee that any explicit statutory articles requested by the user are recovered and promoted to the top of the context sequence, bypassing vector distance limitations.

The consolidated context array is finalized through a two-stage semantic cross-encoder reranker and routed directly into the generation gate. To ensure full compliance with the principle of legislative authority, the synthesis engine applies a *Normative-Rank Context Budgeting Policy*, restricting the volume of technical circular chunks to prevent them from diluting primary statutory codes. The final answer string is synthesized utilizing a conversational *Semantic Template Anchoring* approach (Γ) and passed through a rigorous **Deterministic Validation Gate**. The validator computes the logical conjunction of five discrete boolean guardrails, evaluating inline citation presence, retrieval boundaries, article numeral authenticity, rank priorities, and uncertainty disclosures. If any guardrail is violated, the stochastic text generation is suppressed entirely, and the system programmatically activates the fallback track.

This comprehensive architecture establishes the theoretical and structural foundation for the subsequent sections of Chapter 3. Section 3.2 introduces the specialized legal corpus and document metadata characteristics. Section 3.3 delineates the text cleaning,

syntax-aware parsing, and metadata inheritance strategies. Section 3.4 formalizes the conceptual design, node semantics, and edge-typing rule matrix of the Neo4j knowledge graph. Section 3.5 details the formulation of the multi-query hybrid retrieval baselines and reciprocal rank fusion indexing. Section 3.6 formalizes the dual-layered intent routing layers, forced promotions, and adaptive graph expansion policies. Finally, Section 3.7 establishes the semantic cross-encoder harmonization, normative budgeting distributions, and the deterministic 5-tier validation guardrails that secure system output.

3.2 Legal Corpus and Data Sources

The performance, reliability, and precision of a Retrieval-Augmented Generation (RAG) system are fundamentally governed by the scope, structural integrity, and authority of its underlying knowledge base. In the domain of legal question answering, these parameters are especially critical: legal language is highly structured, dense, and semantically sensitive, where minor phrasing variations carry distinct statutory implications. Consequently, retrieval failures or LLM-based hallucinations in legal contexts can lead to incorrect advice, regulatory non-compliance, or severe procedural errors. To mitigate these risks and establish a highly reliable, verifiable QA pipeline, this study utilizes an intentionally scoped legal corpus consisting of six key Vietnamese labor-related documents, summarized in Table 3.1.

Rather than attempting to index the entire, dynamic repository of Vietnamese national legislation—which spans tens of thousands of laws, administrative decrees, and circulars across disjoint regulatory domains—this project deliberately restricts its scope to a representative core of labor law. This intentional scoping serves three main methodological purposes: First, it ensures a highly controlled environment for evaluating the extraction accuracy of legal cross-references and parent-child hierarchical relations, which are critical for graph-based semantic indexing. Second, it allows for exhaustive, domain-expert manual validation of the generated answers and citation paths, enabling the construction of a high-fidelity evaluation benchmark. Third, it directly mirrors the architecture of domain-specific legal expert systems, which prioritize deep, specialized semantic coverage over broad but shallow general knowledge. It is important to emphasize that the proposed system is strictly bounded by this indexed corpus and does not encompass all Vietnamese legislative texts. It is designed as a high-precision, domain-specific advisor for targeted labor-law inquiries and should not be construed as a general legal oracle or a complete substitute for professional legal counsel.

Table 3.1: Summary of the scoped Vietnamese labor law corpus and document metadata.

Document ID	Type	Normative Rank	Chunks
45-2019-qh14	bo_luat	1	697
92-2015-qh13-labor-only	bo_luat	1	159
ngghi-dinh-135-2020-nd-cp	ngghi_dinh	2	41
ngghi-dinh-145-2020-nd-cp	ngghi_dinh	2	520
thong-tu-09-2020-tt-bltdtbxh	thong_tu	3	73
thong-tu-10-2020-tt-bltdtbxh	thong_tu	3	66

A defining characteristic of the Vietnamese legal framework is its strict hierarchical

structure of legal validity, which maps directly to the normative ranks defined in Table 3.1. This hierarchical alignment is governed by the national Law on Promulgation of Legislative Documents [2]. Understanding this hierarchy is essential for both query routing and context synthesis, as lower-level regulations must conform to higher-level statutes, and conflicts are legally resolved in favor of the higher-ranking document.

Normative Rank 1 (Codes and Laws). This rank represents the highest level of domestic statutory authority, containing primary legislative acts passed directly by the National Assembly of Vietnam. In our corpus, these are represented by the type `bo_luat` (code). Under the principle of *lex superior*, these documents establish the foundational legal rights, duties, and frameworks that all subordinate instruments must strictly adhere to.

Normative Rank 2 (Decrees). Promulgated by the Government of Vietnam and signed by the Prime Minister, decrees (represented by the type `ngghi_dinh`) occupy the second level of the hierarchy. The primary function of a decree is to detail, guide, and implement the broad provisions outlined in a Rank 1 code. While subordinate to codes, decrees carry substantial administrative authority and represent the main operational directives for government ministries and public enforcement.

Normative Rank 3 (Circulars). Positioned at the third level of the hierarchy, circulars (represented by the type `thong_tu`) are issued by individual ministries or ministerial-level agencies. In this corpus, these documents are promulgated by the Ministry of Labour, Invalids and Social Affairs (MOLISA) to provide highly granular, procedural instructions, administrative forms, and narrow technical specifications required to execute decrees and codes. Under Vietnamese legal practice, circulars must remain strictly within the bounds of their parent codes and decrees.

To construct a comprehensive representation of labor relations and disputes, the selected six documents cover the entire span of the legal hierarchy—from primary statutory rights down to specific administrative templates and procedural litigation rules:

Labor Code 2019. Represented by the identifier 45-2019-qh14, the Labor Code 2019 is the foundational legislative pillar of the entire corpus. Promulgated by the National Assembly on November 20, 2019, and taking effect on January 1, 2021, it establishes the fundamental principles governing labor relations, employment standards, trade unions, social dialogue, and labor dispute resolution. The document comprises 17 chapters and 220 articles, which are preprocessed into 697 distinct text chunks. It acts as the ultimate authority for defining employment contracts, statutory benefits, minimum working age, and occupational rights.

Implementing decrees and circulars. To model the detailed administrative and procedural rules that implement the Labor Code 2019, the corpus integrates four key guiding documents:

- *Decree 145/2020/ND-CP* (`ngghi-dinh-145-2020-nd-cp`): This Rank 2 administrative decree is a massive regulatory document (yielding 520 chunks) that provides comprehensive guidance on several core aspects of the Labor Code, including specific regulations on labor contract conditions, wage calculations, overtime hours, labor discipline, occupational safety, and mandatory policies for female and minor employees. It represents the primary source of operational details for daily business compliance.
- *Decree 135/2020/ND-CP* (`ngghi-dinh-135-2020-nd-cp`): This specialized Rank 2 decree establishes the regulatory roadmap for increasing the national retirement

age in Vietnam. Crucially, it contains the official retirement age lookup tables and scheduling indexes based on the employee’s gender, birth month, and work hazards, preprocessed into 41 chunks.

- *Circular 09/2020/TT-BLDTBXH* (`thong-tu-09-2020-tt-bltdtbxh`): Promulgated by MOLISA, this Rank 3 circular (yielding 73 chunks) details the protective regulations and specific conditions governing employment relations with minor workers under 15 years of age. It includes the official list of permissible light-work occupations and mandatory health-and-safety guidelines for employing youth.
- *Circular 10/2020/TT-BLDTBXH* (`thong-tu-10-2020-tt-bltdtbxh`): This Rank 3 circular (yielding 66 chunks) provides detailed templates and mandatory content requirements for labor contracts, covering job scopes, wage structures, social insurance obligations, and intellectual property clauses.

Labor-related civil procedure provisions. Represented by the document ID 92-2015-qh13-labor-only, this dataset consists of a carefully filtered subset of the Civil Procedure Code 2015 (Law No. 92/2015/QH13), yielding 159 chunks. While the complete Civil Procedure Code governs all civil, commercial, and family lawsuits in Vietnam, our corpus selectively extracts only the specific provisions that govern labor disputes, court jurisdictions, evidentiary requirements, litigation fees, and procedural timelines for labor-related lawsuits. The inclusion of these Rank 1 provisions is vital for query answering because employment-related questions frequently transition from substantive rights (e.g., “What are my rights under unfair dismissal?”) to procedural remedies (e.g., “How and where do I file a lawsuit against my employer?”). By linking litigation procedures directly to substantive labor rights, the corpus enables the QA system to support multi-hop legal reasoning and comprehensive user guidance.

3.3 Legal Document Preprocessing and Enrichment

The fundamental bottleneck of any Retrieval-Augmented Generation (RAG) system operating in specialized domains is the transition from raw, unstructured source documents to highly structured, queryable semantic units. In the legal domain, this transition is particularly demanding. Legal documents are characterized by strict, non-linear hierarchical organizations, which are not merely formatting preferences but are intrinsic to the validity and interpretation of statutory provisions. In Vietnamese legislative drafting, documents follow a deeply nested hierarchy consisting of chapters, sections, articles (*Điều*), clauses (*Khoản*), and points (*Điểm*), often augmented by detailed appendices (*Phụ lục*) and tables (*Bảng*).

In standard RAG implementations, documents are routinely split into flat, fixed-size chunks of a predefined token or character length (e.g., 500 tokens with a 50-token overlap). While computationally simple, this arbitrary segmentation is highly inappropriate for legal texts. Because fixed-size chunking is blind to the structural grammar of a statute, it frequently splits single cohesive clauses across chunk boundaries, resulting in semantic fragmentation. More critically, it separates deeply nested points from their parent clauses and articles. For example, a retrieved chunk that contains only a point stating “*c) Người từ đủ 15 tuổi đến dưới 18 tuổi*” (c) Persons from full 15 to under 18 years of age) is completely uninterpretable without its parent context, which defines this as the age range for minor employment under Article 143. Naive chunking thus destroys the

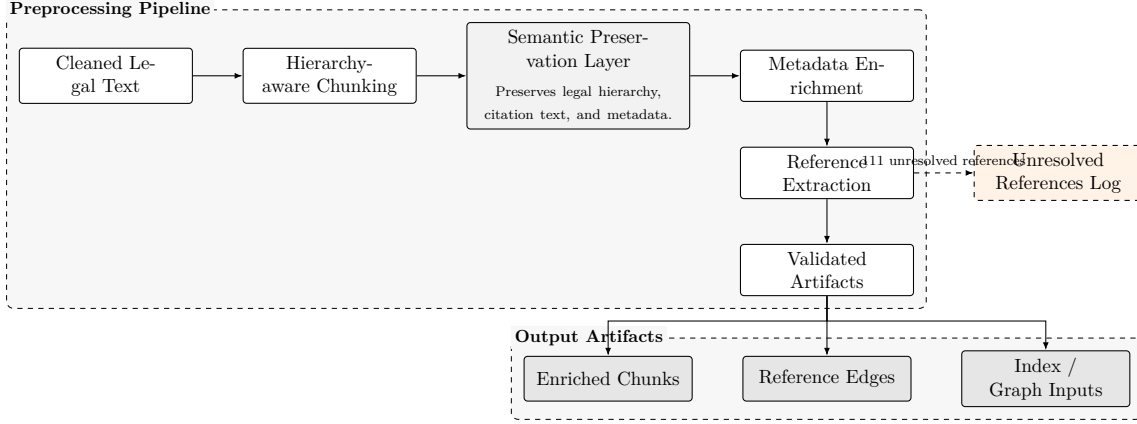


Figure 3.2: Legal data processing pipeline with hierarchy-aware chunking and metadata preservation.

context necessary for dense vector matching and makes it mathematically impossible for a generative language model to construct or validate precise legal citations. To preserve the structural integrity of the law without overclaiming a human-like semantic understanding, our system implements a specialized data processing pipeline (depicted in Figure 3.2). This pipeline maps raw Vietnamese legal texts into structured, metadata-enriched chunks through a multi-stage preprocessing and structural isolation workflow.

Text cleaning. Before structural parsing can occur, raw documents are ingested and subjected to a rigorous cleaning phase. Raw legal texts are converted into normalized Unicode (NFC) encoding to resolve spelling and character inconsistencies common in older Vietnamese PDFs. Document-specific metadata headers, footers, PDF page numbers, watermarks, and optical character recognition (OCR) artifacts are programmatically stripped. Double line breaks and structural indentations are normalized while maintaining the structural paragraph borders. This stage outputs a clean, standardized text stream that serves as the input to the chunking engine, with automated validation reports flagged for any characters outside the standard Vietnamese character set.

Hierarchical chunking. Rather than slicing the text stream at uniform intervals, our chunking engine executes a syntax-aware parser that dynamically isolates legal units based on deterministic, regular-expression-based grammars. These structural grammars are defined to detect key boundaries in the text stream, mapping the document’s nested hierarchy into a parented, object-oriented memory layout. The core patterns utilized by the parser include:

- **ARTICLE_RE:** `^Điều\s+(?P<number>\d+[A-Za-z]?)\.\s*(?P<title>.*)$` — Isolates article numbers and titles, supporting numerical additions (e.g., Article 11a).
- **PART_RE:** `^PHẦN\s+(?P<number>.*)$` — Identifies major legislative parts.
- **CHAPTER_RE:** `^CHƯƠNG\s+(?P<number>[IVXLCDM0-9]+[A-ZĐ]?)\.\s*(?P<title>.*)$` — Identifies Roman-numeral chapter boundaries.
- **SECTION_RE:** `^MỤC\s+(?P<number>[IVXLCDM0-9]+|\d+)\.\s*(?P<title>.*)$` — Extracts legislative sections within chapters.
- **CLAUSE_RE:** `^(?P<label>\d+)\.(?!d)\s+(?P<text>.*)$` — Detects numeric clause borders while avoiding false positives from floating-point decimals.

- `POINT_RE: ^(?P<label>[a-zđ])\)\s+(?P<text>.*)$` — Detects lower-case alphabetical point listings.
- `SUBPOINT_RE: ^(?P<label>[a-zđ](?:\.\d+|\d+)\)\)\s+(?P<text>.*)$` — Detects deep subpoint listings (such as a.1 or b.2).

The inclusion of the `SUBPOINT_RE` pattern is a crucial operational detail: certain dense Vietnamese provisions structure exceptions or granular conditions using subpoint labels. By parsing these explicitly, the system prevents subpoint elements from being merged into generic point texts, maintaining the highest possible layout granularity. This syntax-aware parser processes the document stream to build a lightweight, abstract-syntax-tree-like hierarchy of nested legal provisions, preserving parent headings and layout relationships without making overclaims of semantic legal comprehension.

Semantic preservation. The core innovation of our preprocessing pipeline is the *Semantic Preservation Layer*. This layer is responsible for maintaining the statutory context of nested legal provisions across chunk boundaries through a process of explicit metadata inheritance. When the hierarchical chunker extracts a granular point-level chunk, the Semantic Preservation Layer forces that chunk to inherit structural and citation metadata from its parent clause, article, and document.

Consider the concrete legal provision *Điểm c, Khoản 2, Điều 3* of the Labor Code 2019. Under naive chunking, this point is isolated as a flat string. Under our Semantic Preservation Layer, the resulting chunk explicitly inherits and encapsulates the following structured key-value parameters:

- `point_ref = "c"`
- `clause_ref = "2"`
- `article_number = 3`
- `document_id = "45-2019-qh14"`
- `document_title = "Bộ luật Lao động 2019"`
- `document_type = "bo_luat"`
- `normative_rank = 1`
- `citation_text = "Điểm c Khoản 2 Điều 3 Bộ luật Lao động 2019"`
- `document_hierarchy = "45-2019-qh14/dieu-3/khoan-2/diem-c"`

This metadata inheritance radically improves downstream processes in two ways. First, during retrieval, it allows the vector search to be combined with deterministic metadata filtering (e.g., restricting search to Rank 1 documents or specific articles). Second, during answer generation, the inclusion of the synthesized `citation_text` directly within the chunk payload ensures that the LLM is supplied with the exact, formal legal basis for its statements, preventing the generation of invented or misattributed citations.

A critical architectural decision in our pipeline design is the explicit separation of the chunk’s textual content into two distinct fields: `retrieval_text` and `citation_text`. The `retrieval_text` field is specifically optimized for dense vector semantic matching and embedding calculations. It combines the raw statutory text of the chunk with parent hierarchical context headers (such as the document title, chapter heading, and parent article title). This context injection ensures that the embedding model has access to the complete legal framework, enabling it to compute more accurate semantic similarity scores

even when user queries are highly implicit or abstract. Conversely, the `citation_text` field is designed as the citation surface for final answer generation. It represents the exact, formal legal location (e.g., “Điểm c Khoản 2 Điều 3 Bộ luật Lao động 2019”), providing the precise statutory address. Dividing these fields is a primary architectural decision: `retrieval_text` guarantees high-quality matching during dense vector searches, while `citation_text` supports strict citation grounding and rigorous hallucination control in LLM response generation.

Table 3.2: Legal chunking and data integrity validation results.

Validation Item	Value
Legal chunks	1,556
Duplicate chunk IDs	0
Missing citation text	0
Missing article number	0
Very short chunks	0
Very long chunks	0

Metadata enrichment. Beyond structural attributes, the pipeline enriches each extracted chunk with 19 discrete metadata fields to enable multi-faceted indexing and advanced query routing. These fields, detailed in our database schema, include operational parameters such as `chunk_id`, `chunk_type` (e.g., text, table, or form), and `source_file`.

Crucially, the enrichment pipeline implements a deterministic, rule-based mapping function, `infer_enriched_taxonomy`, to assign legal taxonomy categories without introducing stochastic behavior. The function builds a normalized aggregated context string by joining fields from the chunk’s hierarchy, including `document_title`, `part_heading`, `chapter_heading`, `section_heading`, `article_title`, `appendix_heading`, `heading`, `text`, and `retrieval_text`. This aggregated string is scanned using rule-based dictionaries containing keyword patterns:

- **TOPIC_RULES:** Covers 21 core legal topics (e.g., `hop_dong_lao_dong`, `tien_luong`, `ky_luat_lao_dong`).
- **ACTOR_RULES:** Covers 10 distinct legal entities/actors (e.g., `nguoi_lao_dong`, `nguoi_su_dung_lao_dong`, `toa_an`).
- **ISSUE_TYPE_RULES:** Covers 15 legal issue types (e.g., `giao_ket_hop_dong`, `don_phuong_cham_dut`, `tham_quyen_toa_an`).

Using a rule-based dictionary scan rather than stochastic LLM auto-labeling represents an essential design choice: it guarantees 100% reproducibility, eliminates the risk of stochastic hallucinations in metadata generation, and significantly reduces ingestion latency and computational costs.

Appendix and table handling. Unlike standard text processors that struggle to parse non-paragraph elements and often discard tables as unindexable noise, our pipeline treats appendices and tables as primary structured evidence chunks. This is mathematically and legally essential: in Vietnamese labor law, critical details such as the gender-specific retirement schedules in Decree 135/2020/ND-CP or standard contract templates in Circular 10/2020/TT-BLDTBXH are housed entirely within tables and appendices.

Our parser converts these tables into dense markdown representations, preserving column relationships and ensuring that cell-level lookups are fully searchable. These chunks are embedded and indexed with a dedicated `chunk_type` of "table" or "appendix" and are linked directly to their parent document nodes in our graph database, allowing users to retrieve exact mathematical lookup tables (with visual layout mappings detailed in Appendix G).

To verify the robust performance of the data processing pipeline before database ingestion, the corpus is passed through an automated validation layer. This layer performs structural sanity checks to detect duplicate identifiers, empty payloads, orphan nodes, and anomalous chunk lengths. Crucially, a quantitative pruning criteria is enforced: any chunks with text shorter than a strict character limit of `VERY_SHORT_TEXT_THRESHOLD` = 20 are discarded as fragmented lines or parsing noise, making the validation process quantitative rather than subjective.

As reported in Table 3.2, the preprocessing pipeline of the scoped corpus yielded exactly 1,556 validated legal chunks. The validation run completed with zero duplicate chunk IDs, zero instances of missing citation texts, zero missing article numbers, and zero short or long chunks. This zero-error outcome provides robust scientific verification of our pipeline’s integrity: First, the absence of duplicate chunk IDs indicates stable, deterministic chunk identification across all documents. Second, the zero missing citation text count proves that every single evidence chunk in the database possesses a valid statutory address and can be explicitly cited. Third, the zero missing article number metric demonstrates that the parent article-level hierarchy was successfully preserved for all chunks. Finally, the lack of very short chunks indicates that our parser successfully filtered empty lines and parsing noise below the 20-character threshold without discarding genuine statutory content. While this syntax-aware pipeline does not semantically understand the law in a cognitive manner, it preserves the legislative document structure and citation context, establishing the foundational data layer required for dense vector retrieval, Neo4j graph construction, and precise answer grounding.

3.4 Legal Cross-Reference Extraction and Graph Design

Beyond isolating individual legal passages, a primary requirement for legal reasoning is the capability to navigate the dense web of cross-references and legal hierarchies that bind regulatory documents. Legal texts are rarely read in isolation; a single article in a ministerial circular is frequently qualified by a clause in an implementing decree, which in turn derives its authority from a primary legislative code. To capture these semantic and hierarchical associations, this project designs and constructs a directed legal knowledge graph using the Neo4j graph database. It is important to emphasize that this graph is not a stochastic, LLM-generated entity graph, which is prone to structural noise, missing relations, and hallucinated connections. Instead, it is a deterministic, document-structure legal graph constructed directly from stable legal hierarchies, regular-expression-based citation patterns, explicit cross-references, rule-based taxonomy metadata, and normative legal validity ranks. This deterministic schema ensures 100% logical precision and full structural auditability, both of which are foundational for deploying RAG systems in high-risk regulatory environments.

The conceptual schema of the Neo4j legal knowledge graph is illustrated in Figure 3.3,

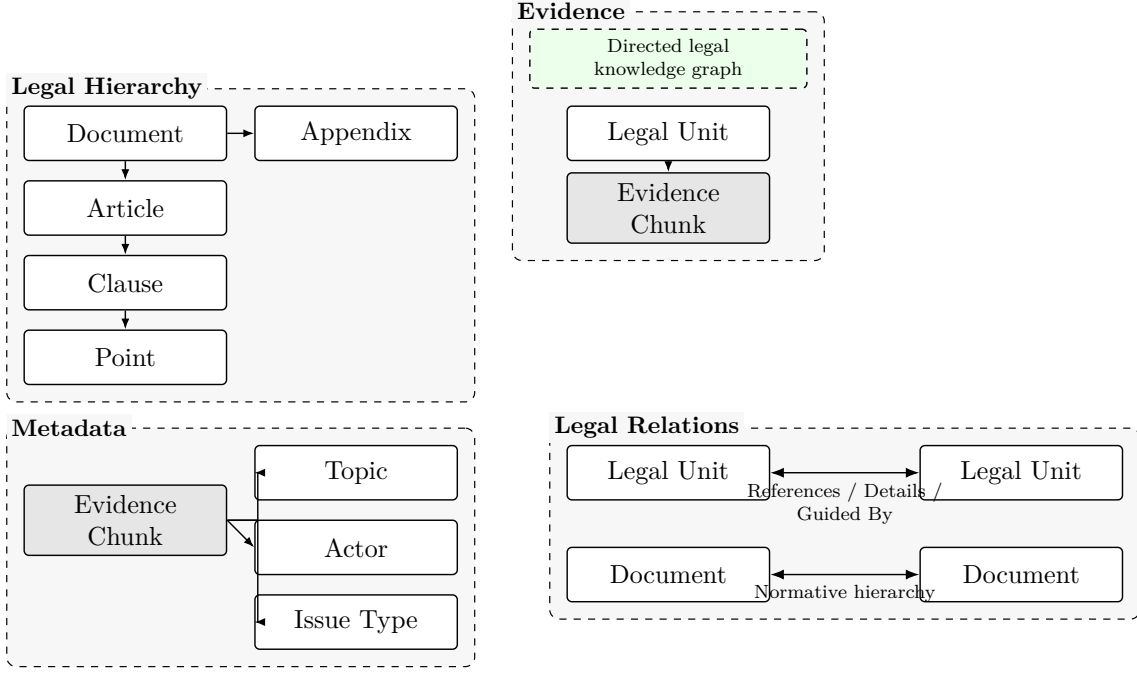


Figure 3.3: Conceptual schema of the Neo4j legal knowledge graph.

which represents the conceptual directed legal graph schema. The graph is organized around nine distinct node labels and fourteen relationship types, capturing the structural, referential, taxonomic, and normative dimensions of the legal corpus.

The architecture of the knowledge graph is divided into four primary areas, each representing a distinct legal relationship domain: First, the **structural hierarchy** defines the internal skeleton of each document. The nodes `Legal_Document`, `Legal_Article`, `Legal_Clause`, `Legal_Point`, and `Legal_Appendix` form the hierarchical backbone of the graph. The relationships `HAS_ARTICLE`, `HAS_CLAUSE`, `HAS_POINT`, and `HAS_APPENDIX` represent this containment taxonomy, mapping the nesting structure directly. The physical text segments are represented by `Evidence_Chunk` nodes, connected to their leaf-level parent nodes via the directed relationships `HAS_SOURCE_CHUNK` and `SOURCE_OF`. Second, the **cross-reference edges** model citation links between different legal provisions. These are represented by three distinct edge types: `REFERENCES` (a general citation between legal units), `DETAILS` (when a lower-ranking provision directly elaborates on a higher-ranking statutory article), and `GUIDED_BY` (when a statutory provision is explicitly subject to ministerial implementation guidance). These edges enable the retrieval engine to perform multi-hop graph expansion, gathering associated context that is geographically distant in the corpus but semantically bound by the law. Third, the **taxonomy edges** connect `Evidence_Chunk` nodes to semantic legal classification nodes—`Legal_Topic`, `Legal_Actor`, and `Legal_IssueType`—using the relationships `MENTIONS_TOPIC`, `APPLIES_TO_ACTOR`, and `HAS_ISSUE_TYPE`. These connections support multi-faceted filtering and query routing during seed retrieval. Fourth, the **normative hierarchy** establishes relationships of legal authority between different regulatory documents. The authority relationships are represented by connecting `Legal_Document` nodes of different ranks using `SUPERIOR_TO` (representing constitutional or legislative superiority) and `MUST_COMPLY_WITH` (representing the requirement that subordinate regulations conform to parent laws). In Vietnamese labor law, this hierarchy follows a strict ranking of Code (`bo_luat` or `luat`) > Decree (`ngghi_dinh`) > Circular (`thong_tu`). Preserving this

Table 3.3: Summary of loaded node and edge counts in the Neo4j legal knowledge graph.

Graph Item	Count
Documents	6
Articles	411
Clauses	1,235
Points	774
Appendices	35
Evidence chunks	1,556
Resolved reference edges loaded	948
Taxonomy edges	6,099
Normative hierarchy edges	20

rank is essential for query routing, prioritizing statutory authority over administrative instructions in conflict resolution.

The loaded statistics of the resulting graph are summarized in Table 3.3, which details the node and edge counts, including 6 documents, 411 articles, 1,235 clauses, 774 points, 35 appendices, 1,556 evidence chunks, 948 resolved reference edges, 6,099 taxonomy edges, and 20 normative hierarchy edges.

The resulting graph forms a dense mathematical network. To build this network, the preprocessing pipeline executes a specialized citation parsing and reference resolution workflow, detailed as follows:

Citation pattern detection. The system analyzes the raw text of each legal chunk using regular-expression-based patterns to detect citation expressions. To handle the varied citation styles in Vietnamese legislative documents, the extraction engine implements three primary regular expressions to scan normalized text: `ARTICLE_REFERENCE_RE` targets standard hierarchical citations containing article numbers, clause indices, and point letters; `MULTI_CLAUSE_REFERENCE_RE` isolates coordinated multi-clause citations within a single article; and `DOCUMENT_REFERENCE_RE` identifies official document citations based on document numbers, years, and ministerial suffixes (such as *Nghị định 145/2020/NĐ-CP* or *Thông tư 10/2020/TT-BLĐTĐ*). These patterns are designed to identify both absolute citations, which explicitly name specific target documents (e.g., “Điều 5 Nghị định số 145/2020/NĐ-CP”), and relative citations, which refer to local or parent structures within the same document (e.g., “Khoản 2 Điều này” or “quy định tại Mục 1 Chương III”). The parser extracts the coordinates (document, chapter, article, clause, point) of each candidate citation.

To handle the syntactic complexity of Vietnamese legislative citations, where multiple clauses may be coordinated in a single sentence, the parser implements a specialized compound reference extractor. Vietnamese legislative citations often aggregate multiple clauses in one expression, such as “khoản 1, 2 và 3 Điều 10”. Instead of treating the whole phrase as a single reference which would collapse multiple legal anchors, the parser handles this using the `MULTI_CLAUSE_REFERENCE_RE` pattern. The extractor parses the coordinated clause sequence using the internal helper `_parse_clause_numbers` to extract the individual numerical indices and programmatically instantiates separate target references for each clause. In this manner, a phrase like “Khoản 1, 2 và 3 Điều 10” is deterministically unpacked into separate references to Khoản 1 Điều 10, Khoản 2 Điều 10, and Khoản 3 Điều 10. This design choice dramatically improves graph granularity,

avoiding the loss of distinct legal citations under a single collapsed edge and enabling targeted multi-hop traversals.

Reference resolution. Once detected, candidate references are passed to a normalization engine to resolve their target node identifiers. Relative citations are mapped to absolute node paths by looking up the parent article and document ID within the structural AST-like tree constructed during preprocessing. Absolute citations are normalized by matching document titles, abbreviations, and numeric expressions against our registry of document aliases (e.g., mapping “Bộ luật Lao động” to 45-2019-qh14).

In Vietnamese legislative drafting, documents frequently utilize localized relative references whose specific targets depend strictly on the source document context, such as “nghị định này” (this decree), “thông tư này” (this circular), or “luật này” (this law). To normalize these references, the resolution engine utilizes a dedicated helper, `_resolve_relative_document_alias`, which leverages the source chunk’s metadata context—specifically its `document_id` and `article_number`—to map such relative phrases to absolute target identifiers. For instance, “nghị định này” inside Decree 145/2020/NĐ-CP is resolved to the document ID of Decree 145. A notable edge-case occurs with the relative alias “luật này” (this law) in Article 219 of the Labor Code 2019. In this specific article, which contains transitional and amending provisions, the expression “Luật này” refers to various external amended laws (such as the Law on Social Insurance or the Civil Procedure Code) rather than the Labor Code itself. In the implemented resolver, Article 219 is handled as a special case to avoid incorrectly resolving amendment-related references as ordinary intra-document citations. The resolver conservatively maps the target to a dedicated external quoted amended-law placeholder node (“`external-quoted-amended-law`”) instead of forcing it into the indexed Labor Code’s internal structure. Once normalized, the target identifiers are cross-checked against our database of active node IDs before reference edges are generated.

Graph schema and edge-typing. When resolved references are loaded into the database, the edge type between the source unit (s) and the target unit (t) is computed dynamically based on their respective legislative normative ranks. Normative ranks are mapped numerically from document metadata, where Rank 1 corresponds to a primary Code or Law, Rank 2 corresponds to an administrative Decree, and Rank 3 corresponds to a ministerial Circular. Thus, a numerically larger rank denotes a lower-level administrative document in the legislative hierarchy (e.g., a Rank 3 circular is subordinate to a Rank 2 decree, which is subordinate to a Rank 1 code). The formal edge-typing rule is defined by the following mapping:

$$\text{EdgeType}(s, t) = \begin{cases} \text{DETAILS}, & \text{if Rank}(s) > \text{Rank}(t), \\ \text{GUIDED_BY}, & \text{if Rank}(s) < \text{Rank}(t), \\ \text{REFERENCES}, & \text{otherwise.} \end{cases}$$

Under this formulation, when a lower-level document cites a higher-level statute (e.g., a Rank 2 decree detailing a Rank 1 code provision), the edge is typed as `DETAILS`, representing structural elaboration. When a higher-level statute points to a lower-level implementing regulation, the edge is typed as `GUIDED_BY`. Same-rank or neutral references (such as an article in the Labor Code citing another article within the same code) are typed as `REFERENCES`.

Furthermore, in the edge-generation pipeline (`build_reference_edge_records`), when a cross-document `DETAILS` edge is generated, the system automatically instantiates a cor-

responding inverse **GUIDED_BY** edge pointing in the opposite direction. A decree or circular often details a provision in the Labor Code. The primary **DETAILS** edge represents the citation direction from the lower-level implementing text (e.g., a circular or decree) to the higher-level statutory provision (e.g., the Labor Code). Instantiating the inverse **GUIDED_BY** edge allows graph traversal from the Labor Code provision back to its implementing guidance. This architectural decision significantly improves graph expansion during retrieval because the traversal engine can navigate both from guidance to source law and from source law to guidance. This bidirectional capability is particularly powerful for resolving multi-hop and document-guidance queries. It should be emphasized that we do not assume all graph relationships are bidirectional; rather, selected inverse edges are generated specifically for cross-document **DETAILS** relations to facilitate this bidirectional traversal capability.

Unresolved references. To maintain scientific honesty and transparency, it should be clarified that the reference parsing and resolution pipeline is not assumed to be perfect, nor does the resulting graph capture all possible legal relationships. Out of a total of 1,059 citation expressions identified by the extraction engine within the scoped corpus, exactly 948 reference edges were successfully resolved and loaded into the Neo4j knowledge graph. The remaining 111 references were logged and intentionally excluded from graph traversal, rather than being treated as system errors. This deliberate omission is driven by a design philosophy that favors conservative graph construction over forcing uncertain or incorrect edges into the active retrieval network. The 111 unresolved references primarily stem from four distinct factors: (1) references to external legislative documents that lie outside the indexed six-document corpus, such as the Civil Code 2015, the Law on Social Insurance, or various tax laws; (2) citation expressions that could not be safely normalized due to incomplete textual coordinates; (3) references to external laws or amended provisions; and (4) ambiguous relative references that lacked specific anchor points (e.g., vague terms like “các quy định liên quan”). By logging and excluding these unresolvable references, the system avoids generating false associations that would contaminate downstream multi-hop traversals, ensuring that the legal knowledge graph remains a high-fidelity representation of the verified corpus.

3.5 Vector-based and Hybrid Retrieval

Prior to executing graph-based context expansion, the system establishes a highly precise baseline retrieval layer utilizing vector-based and hybrid search. In the domain of legal question answering, relying solely on a single retrieval modality is highly inadequate. While dense semantic search is capable of identifying conceptual synonyms, it is notoriously prone to lexical drift and struggles to match exact statutory citations or numerical coordinates. Conversely, sparse keyword matching is exceptionally precise at capturing exact article numbers and legislative IDs but is completely blind to paraphrased user queries. To address these individual limitations, this project implements a unified hybrid retrieval framework, utilizing the Qdrant vector database [5] to index and query the preprocessed legal corpus.

The baseline retrieval engine operates over exactly 1,556 hierarchy-aware legal chunks. During database ingestion, each chunk is encoded into both dense and sparse vector representations. The dense semantic embeddings are generated using the `keepitreal/vietnamese-sbert` model [7], a Sentence-BERT transformer model optimized for Viet-

namese natural language understanding, which projects each chunk into a 384-dimensional dense vector space. For sparse lexical matching, the chunk text is processed using a sparse encoder to generate term-importance vectors, capturing structural keywords and statutory terminology. Crucially, each vector index entry preserves a comprehensive payload of legal metadata, including: `chunk_id`, `document_id`, `article_number`, `clause_ref`, `citation_text`, `document_type`, `normative_rank`, `topic`, `actor`, `issue_type`, and the absolute hierarchical directory path. This payload preservation is a critical architectural choice, as it allows the retrieval engine to execute metadata-prefiltered searches and apply legal intent-specific constraints during query execution.

To execute a search, the system does not simply compute the similarity score of the raw user query. As illustrated in the overall system flow (Figure 3.1), the query first passes through an intent classifier which identifies target documents, candidate article references, and key actor or issue tags. The system then generates multiple semantic query variations using the internal helper `build_query_variants`. This step is essential to bridge the lexical gap between a user’s conversational query and the formal statutory phrasing of Vietnamese national regulations. The generated set of query variants, V , can include the raw user query, rule-based query expansions, issue-focused hints, forced citation-oriented variants, and article/citation coordinates. For each query variation, the retrieval engine encodes both dense and sparse vectors, executing parallel prefetch queries in Qdrant.

This multi-query, multi-modal parallel prefetching architecture is designed to reduce embedding misalignment by avoiding reliance on a single vector representation of the user query. The total number of parallel prefetch channels, $|P|$, executed for a single query intent is determined by the following formal relation:

$$|P| = 2|V| + \mathbb{I}(\text{ref_boost}) + \mathbb{I}(\text{issue_focus})$$

where V is the set of generated query variants, and P is the set of Qdrant prefetch channels. The factor $2|V|$ represents the concurrent execution of one dense prefetch channel (using the dense vector registry) and one sparse prefetch channel (using the sparse vector registry) for each generated query variant. The term $\mathbb{I}(\text{ref_boost})$ is an indicator function that takes the value 1 when a deterministic reference-boost filter is active, and 0 otherwise. Similarly, $\mathbb{I}(\text{issue_focus})$ is an indicator function that takes the value 1 when an issue-focus filter is active, and 0 otherwise.

In the search execution flow implemented in `qdrant_search.py`, both the reference-boost filter (`ref_boost`) and the issue-focus filter (`issue_focus`) are routed exclusively through the sparse vector prefetch channel. This is a critical design choice: because article numbers, document IDs, `issue_type` labels, and legal actor entities are represented as deterministic legal tokens and discrete payload fields, they are more reliably matched by sparse lexical indexing combined with payload pre-filtering. By routing these high-precision legal-coordinate boosts through the sparse vector channel rather than relying on dense semantic similarity, the searcher successfully reduces dense semantic noise, preventing exact legal coordinates from being blurred by superficial embedding similarities.

Once the candidate prefetch lists are returned from all active prefetch channels, the search engine merges and reranks the candidates using Reciprocal Rank Fusion (RRF). In the search implementation, this fusion operation is delegated to Qdrant’s built-in RRF operator. Conceptually, the RRF ranking score for a candidate document or chunk d is

computed by the following mapping:

$$\text{RRF}(d) = \sum_{m \in M} \frac{1}{k + r_m(d)}$$

where M represents the set of active prefetch channels, $r_m(d)$ is the ordinal rank of candidate d within prefetch channel m (with $r_m(d) = \infty$ if d is absent from retrieval list m), and k is a smoothing constant that reduces the dominance of top-ranked items in any single prefetch list. Because the fusion operation is delegated directly to Qdrant’s native RRF query engine, the formula above describes the ranking principle, while the exact smoothing behavior and the specific value of k follow the backend implementation without making unverified configuration assumptions.

This hybrid search architecture is specifically motivated by the distinct and complementary roles of dense and sparse retrieval in the legal domain. Dense retrieval is optimized to capture semantic similarity, conceptual relationships, and conversational paraphrasing, mapping user-centric queries to their legal equivalents. Conversely, sparse retrieval is optimized to capture exact legal tokens, such as specific article numbers, decree codes, circular identifiers, issue labels, and citation-like expressions. Legal question answering fundamentally requires both: legal questions frequently contain deterministic symbols and legal references that must be matched with absolute precision, and which should not be blurred by dense embedding similarities alone.

This hybrid approach directly addresses the severe limitations of dense vector-only retrieval in legal QA. Under a vector-only search, several critical failure cases routinely occur. First, users often phrase their practical queries without mentioning exact statutory articles (e.g., asking about maternity leave durations without referencing Article 139 of the Labor Code). Second, the relevant implementing decree or ministerial circular may not share obvious surface terms with the query, causing dense embeddings to miss them entirely due to semantic drift. Third, crucial appendices and tables—such as the retirement age indexes in Decree 135—are highly specialized lexically and mathematically, placing them far from conversational queries in a dense embedding space. Finally, complex document-guidance and multi-hop legal questions require following structural relationships and legislative hierarchies that cannot be captured by mere embedding similarity. Fusing dense semantic generalization with sparse lexical precision ensures that exact coordinates (such as “Điều 219”, “Nghị định 145”, or “Thông tư 10”) are guaranteed to match, while semantic coverage is maintained.

Nonetheless, while hybrid retrieval represents a substantial improvement over vector-only baselines, it is still fundamentally limited for complex legal reasoning: because it operates over isolated text blocks, it remains oblivious to the structural and referential bonds among legal provisions. For example, if a primary code article (e.g., Article 143 on minor workers) explicitly references a ministerial circular for permissible occupations (e.g., Circular 09/2020/TT-BLĐTBXH), a hybrid search can retrieve the Labor Code chunk but has no way of dynamically traversing the graph to retrieve the circular’s permitted lists unless the query itself explicitly contains the circular’s terms. Although hybrid retrieval successfully gathers individual, high-scoring candidate documents, it treats them as isolated vectors and remains oblivious to the structural and referential bonds between them. To answer complex multi-hop regulatory queries, the system must navigate these statutory cross-references (e.g., moving from a primary law to an implementing decree, and down to a detailed administrative appendix) to construct a legally sound and comprehensive context. In this system, hybrid retrieval is utilized to provide a set of highly

precise seed chunks, S_k . The system then executes query-r

3.6 Query Routing and Graph Expansion Policy

As motivated in Section 3.5, while hybrid search provides strong baseline seed chunks, resolving complex legal questions requires traversing statutory cross-references, taxonomic connections, and legislative hierarchies. In this implementation, the query routing layer acts as the primary analytical engine that translates a raw, conversational user query into a structured, formal **QueryIntent** control object. This object dynamically guides downstream retrieval stages, including candidate seed pre-filtering, forced reference promotion, coordinate fallback, and adaptive context windowing, which collectively enrich the retrieval context before generation. Specifically, hybrid retrieval provides the seed chunks, query routing provides the legal intent and constraints, and graph-based reference expansion fills the missing legal context.

Intent resolution. The query routing layer executes a dual-layer routing process consisting of a structured LLM-based query metadata extractor that resolves semantic intent, and a deterministic, rule-based validation and expansion layer that guarantees compliance with the statutory domain. To extract structural intent, the system leverages a structured language model as a zero-shot metadata extractor. The extraction process is defined in `query_router.py`, where the system instantiates a structured Pydantic model named **QueryMetadata**. This model structures the query’s legal context into nine discrete fields: **actor** (the primary legal actor), **actors** (all relevant legal actors), **topics** (valid legal topic classifications), **issues** (specific legal issues), **document_ids** (explicitly mentioned legal document IDs), **query_types** (the question shape, such as comparison or definition), **article_numbers** (explicit article coordinates), **clause_refs** (explicit clause indices), and **point_refs** (explicit point coordinates). In the `analyze_query_smart` function, the system calls a configured router model (defaulting to `llama-3.1-8b-instant` through the configured API provider) with the temperature set strictly to 0 to minimize stochastic variance. The model’s raw text generation is strictly constrained by a JSON schema constructed via `query_metadata_json_schema`. The schema-constrained output reduces free-form variability, while the subsequent sanitization layer prevents out-of-vocabulary labels from entering the retrieval pipeline. The LLM is not used to generate legal answers at this stage. It is used only as a structured metadata extractor for routing.

Metadata sanitization. Once the structured JSON payload is returned by the LLM, the system routes it through a rigorous, deterministic cleaning function named `sanitize_query_metadata_payload`. Because language models are prone to generating hallucinated or slightly mismatched labels, the sanitization layer normalizes and filters all extracted metadata against our valid domain vocabularies: **VALID_ACTORS**, **VALID_TOPICS**, **VALID_ISSUES**, **VALID_DOCUMENTS**, and **VALID_QUERY_TYPES**. Formally, for any extracted metadata collection, the sanitization operation resolves valid labels through the following mapping:

$$L_{\text{valid}} = \{ l \in L_{\text{extracted}} \mid \text{Normalize}(l) \in \Omega \}$$

where $L_{\text{extracted}}$ is the set of raw labels extracted by the LLM, Ω represents the corresponding set of valid domain labels, and the function $\text{Normalize}(l)$ represents a deterministic Unicode and case normalization pipeline. Under the helper function `_normalize_label`, the string is converted to normalized Unicode (NFKD), accents and combining characters are stripped, casing is standardized to lowercase, and spaces or hyphens are mapped

to underscores (e.g., mapping “người lao động” to `nguoi_lao_dong`). If the normalized label matches a predefined alias inside `LABEL_ALIASES`, it is replaced with the canonical system label; otherwise, if the normalized label does not exist inside the valid vocabulary Ω , it is discarded. This metadata sanitization provides a deterministic safeguard, ensuring that all downstream vector pre-filters and rule-based query expansions receive valid, validated coordinates, which helps prevent stochastic errors from contaminating the database retrieval pipeline. This process makes the routing output compatible with deterministic filters and rule-based expansions.

QueryIntent construction. After the metadata is sanitized, the system executes `query_intent_from_metadata` to construct the unified `QueryIntent` control object. Rather than relying solely on the LLM’s classification, the construction function dynamically merges the sanitized LLM metadata with keyword matches extracted directly from the normalized query stream using `collect_keyword_matches`, rule-based routing and coordinates retrieved from `collect_rule_based_routing`, and mapped article coordinates resolved via `collect_mapped_article_expansions`. The resulting `QueryIntent` contains `actor_filters`, `topic_filters`, `issue_filters`, `document_filters`, `explicit_article_numbers`, `inferred_article_numbers`, `force_reference_article_numbers`, `forced_references`, `clause_refs`, `point_refs`, `query_expansions`, and `query_types`. The `QueryIntent` represents the central control object that links query understanding to retrieval execution, ensuring that subsequent retrieval layers are guided by a unified, structured intent.

Forced reference promotion. When a user query matches high-precision legal rules, the routing engine identifies specific statutory provisions as mandatory contexts that must be retrieved, regardless of their vector similarity score. These are represented as high-confidence forced references inside the set of forced references $\mathcal{R}_{\text{forced}}$. In `reference_expander.py`, the system leverages the helper `forced_reference_records` to fetch records by legal coordinates using the record store helper `fetch_records_by_reference`. To guarantee that these legally required documents are prioritized, the system executes `append_forced_reference_hits` to promote these records, and then invokes `pin_forced_reference_hits` to pin forced-reference matches before other hits. To mathematically enforce this promotion without breaking the downstream reranking pipeline, the score for each promoted forced-reference chunk d_{forced} is adjusted using the following formula:

$$\text{Score}(d_{\text{forced}}) = \max_{h \in S_k} \text{Score}(h) + \Delta_{\text{forced}} - \rho \cdot 10^{-4}$$

where S_k is the set of seed chunks returned by the hybrid vector search, $\max_{h \in S_k} \text{Score}(h)$ is the maximum relevance score among those baseline seeds, Δ_{forced} is the forced-reference score margin (configured as a positive constant), and ρ represents the ordinal rank of the forced record. This score adjustment guarantees that every mandatory forced-reference chunk receives a score higher than the highest-scoring vector search hit, ensuring that these high-confidence legal coordinates are promoted to the top of the retrieval context. This policy is reserved strictly for high-confidence legal coordinates, not arbitrary graph expansion, providing a deterministic safeguard. This policy ensures that legally mandatory anchors are not lost when vector similarity alone ranks them too low.

Coordinate fallback. Beyond explicit forced references, the system implements a robust, deterministic coordinate fallback pipeline via the helper `append_reference_fallback_hits`. During retrieval, the user’s query intent may require specific statutory articles, yet the baseline hybrid search might omit them due to vector space limitations

or semantic gaps. The expander compares the required article coordinates in the query intent (A_{required}) against the article numbers actually present in the pre-filtered hybrid search hits ($A_{\text{retrieved}}$). The set of missing target articles, A_{missing} , is computed via:

$$A_{\text{missing}} = A_{\text{required}} \setminus A_{\text{retrieved}}.$$

If A_{missing} is non-empty, the system executes a record-store fallback operation: it fetches the missing statutory chunks directly from the record store using their absolute article coordinates. If the fallback is triggered by an explicit mention of an article number in the user’s query (explicit fallback), the fetched chunks are promoted slightly above the current hits, receiving a score boost of $+10^{-3}$ relative to the maximum existing score. If the fallback is triggered by rules that infer article numbers from topic/issue mappings (high-confidence inferred fallback), the chunks are fetched and integrated more conservatively with a minor score penalty of -10^{-3} relative to the maximum score. Explicit fallback is used when the user directly mentions an article number, whereas high-confidence inferred fallback is used when rules infer an article from topic/issue mapping. The explicit fallback receives a stronger score adjustment, while the inferred fallback is more conservative to avoid context contamination. This design provides a deterministic safeguard: explicit fallback ensures that when a user asks for a specific article, that article is guaranteed to be retrieved, whereas inferred fallback is kept more conservative to avoid context contamination while still improving coverage for high-confidence legal coordinates.

Adaptive context window. To balance retrieval coverage and prompt noise, the system implements an adaptive context windowing policy inside the reference expander. When a query involves highly complex legal issues or comparative intent, a narrow context window may truncate critical legislative provisions, whereas a broad window for simple lookups introduces unnecessary noise that degrades the generation quality. To resolve this, the system dynamically scales the primary article retrieval limit, L_{primary} , using the following adaptive relation:

$$L_{\text{primary}} = \begin{cases} 8, & \text{if } T_q \cap \Theta_{\text{complex}} \neq \emptyset \text{ or } I_q \cap \Phi_{\text{complex}} \neq \emptyset, \\ 4, & \text{otherwise.} \end{cases}$$

where T_q is the set of query types extracted in the intent, I_q is the set of active issue filters, and Θ_{complex} and Φ_{complex} represent the configured sets of complex query types (such as comparison, procedure, or multi-hop) and complex issues (such as benefit calculations or disputes). For simple queries, the limit defaults to 4 articles, while secondary article limits are kept in a smaller range, typically 2 to 4. This adaptive limit increases recall for complex legal questions while controlling prompt noise for simple lookups, without overclaiming that this always guarantees complete statutory evidence.

Final context assembly. As illustrated in the graph-augmented retrieval flow (Figure 3.4), Qdrant and Neo4j are connected through shared chunk identifiers. Because the seed chunks S_k from the hybrid retrieval layer share identical coordinates with graph nodes, they are directly mapped into graph nodes for Neo4j traversal. The system then traverses the knowledge graph to gather topologically associated legal provisions—such as parent-child structural units, cross-document circular details, or normative hierarchy links. The final retrieved context set C_{final} is consolidated by merging the baseline seed chunks S_k , the graph-expanded context C_{graph} , and the coordinate fallback chunks C_{fallback} (which includes both explicit and inferred fallback hits). The final selected context set is formally resolved using a semantic reranking operation:

$$C_{\text{final}} = \text{Rerank}(S_k \cup C_{\text{graph}} \cup C_{\text{fallback}}).$$

This final context set is passed to the grounded answer generator, where answer construction and citation validation are discussed next in Section 3.7.

3.7 Grounded Answer Generation with Legal Citations

The final stage of the methodology converts the consolidated, reranked retrieval context into a precise, citation-grounded answer. In high-risk regulatory applications, directly exposing raw language model outputs introduces severe compliance hazards due to text generation volatility, semantic hallucination, and citation misattribution. To enforce absolute adherence to the underlying statutory framework, the answering layer implements a closed-loop synthesis pipeline comprising three defensive components: a two-stage hybrid rank-fusion semantic reranker, a normative-rank context budgeting matrix, and a deterministic rule-based validation gate. If the generated payload violates any analytical constraints, the framework programmatically triggers a deterministic fallback mechanism to guarantee information integrity.

Semantic reranking and context harmonization. Before evidence selection occurs, the pooled retrieval candidates undergo an intensive semantic harmonization stage defined in `semantic_reranker.py` [6]. While vector similarity models excel at broad data grouping, they lack the multi-sequence attention granularity required to detect fine-grained legal contradictions. To resolve this, candidate hits within a parameterized bottleneck ($N = \text{top_n} = 24$) are evaluated against user query q using a cross-attention transformer model. The cross-encoder processes query-context string pairs, where the context layout defaults to the preprocessed `dense_text` or fallback concatenated payload configurations (`citation_text || raw_text`) [6]. Rather than substituting the baseline retrieval indices entirely, the engine preserves the original search hierarchy. It computes the ordinal rank of each chunk within the initial hybrid pipeline ($R_{\text{heuristic}}(d)$) and its rank within the cross-encoder score distribution ($R_{\text{semantic}}(d)$), fusing them via a secondary Reciprocal Rank Fusion (RRF) operator:

$$\text{RRF}_{\text{fused}}(d) = \frac{1}{K_{\text{RRF}} + R_{\text{heuristic}}(d)} + \frac{1}{K_{\text{RRF}} + R_{\text{semantic}}(d)}$$

where K_{RRF} is the rank-stabilization constant configured at 60.0. Chunks are sorted in descending order of $-\text{RRF}_{\text{fused}}(d)$, harmonizing baseline topological data with intensive cross-sequence semantic validation before context ingestion.

Normative-rank context budgeting. Following reranking, the framework executes the `select_answer_evidence_contexts` algorithm in `generation.py` to carve out the definitive evidence window. Statutory provisions carry distinct normative weights dictated by their authority tier in the legislative hierarchy: Rank 1 corresponds to primary Codes (`bo_luat`), Rank 2 represents administrative Decrees (`ngghi_dinh`), and Rank 3 denotes ministerial Circulars (`thong_tu`). To prevent granular technical guidelines or specific administrative forms from drowning out primary statutory boundaries, the algorithm enforces a strict per-rank containment budget limit denoted as $B(r)$:

$$B(r) = \begin{cases} 3, & \text{for } r = 1 \text{ (Codes/Laws),} \\ 2, & \text{for } r = 2 \text{ (Decrees),} \\ 2, & \text{for } r = 3 \text{ (Circulars).} \end{cases}$$

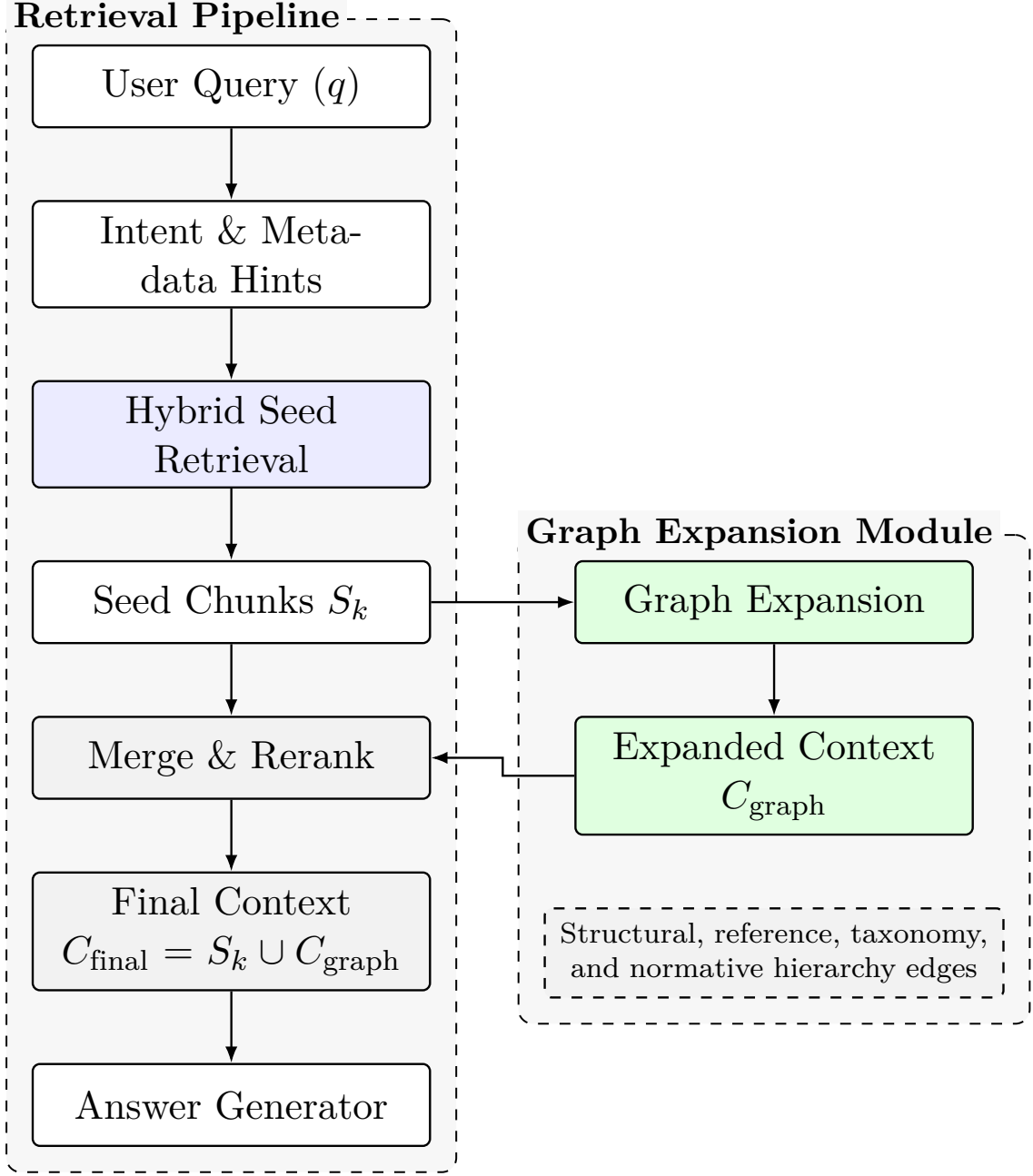


Figure 3.4: Graph-augmented retrieval flow combining hybrid seed retrieval and legal graph expansion.

Furthermore, contexts are dynamically deduplicated by their unique structural article key, $K_{\text{article}} = \langle \text{document_id}, \text{article_number} \rangle$. This structural bottleneck ensures that multiple chunks stemming from a single legal provision cannot exhaust the allocated budget tier, maximizing document variety while controlling prompt size and ensuring that the final generation matrix strictly favors high-authority legislative acts.

Citation-aware answer construction. Grounded answers are generated using the budgeted contexts via the functions `generate_grounded_answer` and `build_extractive_answer_payload`. When an external generator is active, the answering layer performs structured generation by forcing a highly constrained response format via `ANSWER_JSON_SCHEMA`. The synthesized payload is parsed into five discrete properties: `answer` (the natural language response), `legal_basis` (canonical citations), `evidence_quotes` (direct source text pairings), `insufficient_context` (a boolean uncertainty indicator), and `notes`. To guarantee structural continuity and legal consistency, the answering layer enforces a conversational structural lead sentence via a specialized **Semantic Template Anchoring** mechanism (Γ). The operation is formally expressed as:

$$\text{Lead}_{\text{answer}} = \Gamma(q, c_1),$$

where q is the user query, and c_1 represents the highest-priority budgeted context hit. The anchoring function Γ identifies structural token signatures within q (such as retirement or dispute indicators) and maps them to localized formal legal transitions, ensuring the final output flows from a primary act down to subordinate implementing guidance.

Deterministic validation gate. Before any generated answer is returned to the user, it must pass a deterministic validation gate defined by the function `validate_grounded_answer` in `validation.py`. The validation gate evaluates system safety by computing the logical conjunction of five discrete boolean guardrails:

$$\Phi_{\text{validation}} = \phi_{\text{citation}} \wedge \phi_{\text{retrieved}} \wedge \phi_{\text{article}} \wedge \phi_{\text{rank}} \wedge \phi_{\text{uncertainty}}.$$

Each discrete check is mathematically formalized as follows:

- **Inline Citation Guard** (ϕ_{citation}): Verifies that the natural language response string explicitly contains inline citations referencing the designated legal basis array under valid operational states:

$$\phi_{\text{citation}} = \mathbb{I}(\text{Answer contains citation}) \vee \mathbb{I}(\text{insufficient_context} = \text{true})$$

- **Retrieval Boundary Guard** ($\phi_{\text{retrieved}}$): Ensures that every citation listed in the answer’s legal basis can be successfully resolved against the retrieved baseline via canonicalization checks (`canonicalize_citation`), where $\mathcal{C}_{\text{retrieved}}$ represents the retrieved context space, blocking out-of-bounds references:

$$\phi_{\text{retrieved}} = \mathbb{I}(\text{Citations} \in \mathcal{C}_{\text{retrieved}})$$

- **Article Numeral Verification** (ϕ_{article}): Extracting statutory boundary markers via deterministic regular expressions, this gate blocks coordinate hallucinations by ensuring the set of all article numbers mentioned within the answer text (A_{answer}) and the legal basis (A_{basis}) is a strict subset of the valid article entities present within the underlying retrieved contexts (A_{contexts}):

$$\phi_{\text{article}} = \mathbb{I}(A_{\text{answer}} \cup A_{\text{basis}} \subseteq A_{\text{contexts}})$$

- **Normative Rank Priority Gate (ϕ_{rank}):** Evaluates compliance with hierarchical ranking distributions. Let $r_{\text{available}}^{\min}$ be the highest-authority (minimum numeric) rank available in the retrieved contexts, and let $r_{\text{cited}}^{\min}$ be the highest-authority rank cited in the answer’s legal basis. The answer is legally forbidden from skipping primary codes in favor of lower-level guidelines, formulated as:

$$\phi_{\text{rank}} = \mathbb{I}(r_{\text{cited}}^{\min} \leq r_{\text{available}}^{\min})$$

- **Uncertainty Disclosure Gate ($\phi_{\text{uncertainty}}$):** If the context is flagged as insufficient, this gate verifies that the text body explicitly contains predefined uncertainty expressions (`INSUFFICIENT_CONTEXT_HINTS`) to maintain domain trustworthiness:

$$\phi_{\text{uncertainty}} = \mathbb{I}(\text{Answer contains uncertainty}) \vee \mathbb{I}(\text{insufficient_context} = \text{false})$$

If any guardrail fails ($\Phi_{\text{validation}} = 0$), the stochastic response is completely suppressed, and the system programmatically activates the fallback track.

Extractive fallback. To ensure operational stability, if the structured model generation fails to output valid JSON or is rejected by the validation gate while `fallback_on_invalid` is active, the system executes a deterministic extractive fallback. This mechanism leverages the structural layout established during the syntax-aware chunking stage (Section 3.3). Because every chunk explicitly encapsulates its parent metadata and canonical statutory address, the fallback pipeline is able to assemble an unalterable legal response directly from the pre-selected context payloads. It builds the text body using high-relevance paragraph snippets (`_best_rule_snippet`) filtered of low-information lines under an 8-token threshold, binding them directly to their native citation surfaces. This closed-loop design ensures full reproducibility of the system’s evaluation, providing a deterministic reliability check optimized for strict verification within the constructed domain corpus.

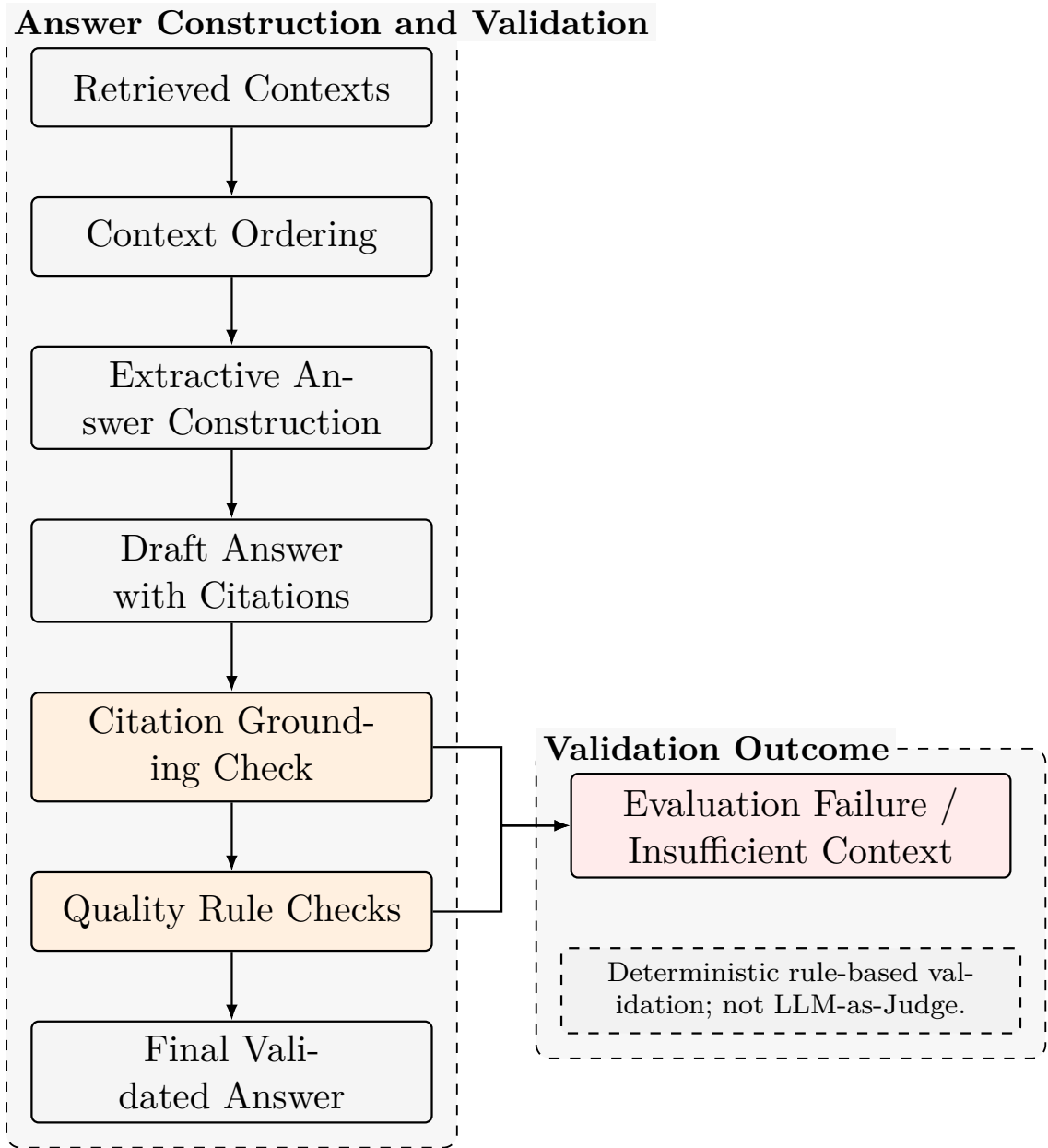


Figure 3.5: Answer generation and deterministic citation validation flow.

Chapter 4

Implementation

4.1 Technology Stack

TODO: Move old implementation content here and update it for the actual project stack. Demote frontend, deployment, and CI details unless they are necessary for reproducing the evaluated RAG pipeline.

Table 4.1: Technology stack.

Layer	Technology	Purpose
Backend	Python, FastAPI	API and orchestration
Embeddings	Sentence Transformers	Dense multilingual vectors
Vector database	Qdrant	Dense and sparse retrieval
Graph database	Neo4j	Legal hierarchy and references
Evaluation	Python scripts and generated artifacts	Benchmark reporting

4.2 Data Processing Implementation

TODO: Document the implemented scripts and modules that transform cleaned legal text into enriched chunks, reference edges, vector records, and graph artifacts.

TODO: Clean text input. Describe cleaned corpus inputs and validation artifacts from artifacts/validation.

TODO: Chunk generation. Describe chunk generation modules and scripts, including hierarchy detection and output files `legal_chunks.jsonl` and `legal_chunks_summary.md`.

TODO: Metadata enrichment. Describe enrichment rules for topic, actor, issue type, document type, normative rank, and citation metadata.

TODO: Cross-reference extraction. Describe reference extraction outputs, including resolved and unresolved reference reports.

4.3 Vector Index and Hybrid Retrieval Implementation

TODO: Move old vector database implementation content here and align it with the Qdrant collection used in the final evaluation.

TODO: Qdrant collection. Document collection `vietnamese_labor_law_chunks`, vector dimension 384, cloud storage mode, and validation that 1,556 chunks were indexed.

TODO: Dense and sparse retrieval. Explain dense vector search, sparse token search, and how hybrid retrieval is implemented. Add key script and module names during chapter writing.

TODO: Metadata payload. List payload fields and explain how they support filtering, citation display, reranking, and validation.

4.4 Neo4j Graph Implementation

TODO: Move old Neo4j graph database details here and keep lower-level implementation details separate from the methodology chapter.

TODO: Node types. Document implemented node types: Document, Article, Clause, Point, Appendix, EvidenceChunk, Topic, Actor, and IssueType.

TODO: Relationship types. Document structural, taxonomy, cross-reference, and normative hierarchy relationships. Include REFERENCES, DETAILS, and GUIDED_BY.

TODO: Graph constraints and indexes. Document Neo4j constraints, indexes, upsert strategy, and validation queries used during graph build. Move verbose schema listings to Appendix C.

4.5 Graph-Augmented Retriever Implementation

TODO: Describe implemented retriever modules and how they use Qdrant records plus Neo4j expansion without turning this chapter into API documentation.

TODO: Seed chunk selection. Explain seed retrieval from hybrid search, including top-k, prefetch limit, metadata hints, and query routing outputs.

TODO: Graph expansion. Explain graph expansion from seed chunks through structural, cross-reference, taxonomy, and normative hierarchy edges.

TODO: Reranking and deduplication. Explain score merging, deduplication by chunk ID, context ordering, and maximum context selection for answering.

4.6 Grounded Answer Generator and Validation

TODO: Move old prompt design content here but revise it to match the implemented grounded extractive answer generator and validation layer.

TODO: Extractive answer generation. Describe the deterministic extractive provider used in the final evaluation and how answer text is selected from retrieved evidence.

TODO: Legal citation formatting. Document legal-basis formatting, citation surfaces, evidence quotes, and source display.

TODO: Normative hierarchy handling. Explain how the answer generator handles higher-rank provisions, implementing guidance, and appendices without implying universal legal interpretation.

4.7 Evaluation Scripts and Reproducibility

TODO: Document scripts for retrieval-mode evaluation, end-to-end evaluation, and final thesis evaluation reports. Reference the generated artifacts in `artifacts/evaluation`. Include environment variables, Qdrant and Neo4j setup, and commands from `artifacts/evaluation/reproducibility_notes.md`. Move long command listings to Appendix F.

4.8 Example Use Cases

TODO: Move old example-use-case content here. Use the case studies requested in Chapter 5 as the main examples: minor worker, retirement age table lookup, labor contract content with circular guidance, and dismissal dispute with court procedure.

Chapter 5

Evaluation and Results

5.1 Evaluation Setup and Benchmark Design

TODO: Introduce the evaluation package and reference `artifacts/evaluation/final_thesis_evaluation_report.md`. Do not add metrics beyond the generated artifacts.

TODO: Corpus statistics. Summarize corpus, chunk, vector index, and graph statistics from the final evaluation report.

Table 5.1: Corpus and index statistics.

Item	Value
Corpus documents	6
Legal chunks	1,556
Vector index count	1,556
Neo4j documents	6
Neo4j articles	411
Neo4j clauses	1,235
Neo4j points	774
Neo4j appendices	35
Neo4j evidence chunks	1,556
Resolved reference edges loaded	948
Taxonomy edges	6,099
Normative hierarchy edges	20

TODO: Benchmark construction. Describe construction of the 69-query benchmark, including query categories, required citation labels, forbidden citation checks, graph-required labels, difficulty labels, and normative-hierarchy-required labels.

TODO: Evaluation environment. Document the evaluation environment from reproducibility notes: PowerShell on Windows, repository virtual environment, sentence-transformer embedding provider, Qdrant collection, Neo4j graph, top-k settings, and output artifacts.

TODO: Curated benchmark design. Describe the curated benchmark without implying it is exhaustive or expert-certified. Cover query categories, required citation labels, forbidden citation checks, and difficulty levels in prose or a compact table. The final benchmark contains 15 easy, 32 medium, and 22 hard queries.

5.2 Retrieval Evaluation Methodology

TODO: Define the retrieval evaluation as a comparison among vector-only, hybrid, and graph-augmented retrieval on the same 69-query benchmark.

TODO: Retrieval modes. Describe vector-only retrieval, hybrid retrieval, and graph-augmented retrieval. Explain that graph-augmented retrieval starts from seed results and expands through the legal graph.

TODO: Retrieval metrics. Define Recall@5, Recall@10, Precision@5, Precision@10, MRR, required citation coverage, and forbidden citation violation.

5.3 Answer and Citation Evaluation Methodology

Answer quality is evaluated using deterministic rule-based validation, not human legal expert review or LLM-as-Judge evaluation.

TODO: Deterministic rule-based validation. Explain the implemented deterministic checks and avoid using RAGAS or LLM-as-Judge terminology as final evaluation methodology unless clearly marked as future work.

TODO: Direct answer and legal rule checks. Explain how the validator checks that an answer addresses the question and includes the required legal rule and required citations.

TODO: Low-information answer detection. Explain how low-information quotes are detected. The final evaluation reports low-information quotes: 0.

TODO: Citation grounding. Explain that answer citations must be supported by retrieved contexts and mapped to known legal citation surfaces.

TODO: Unsupported and unretrieved citations. Explain unsupported article number detection and unretrieved citation detection. The final evaluation reports unsupported article numbers: None and unretrieved citations: None.

5.4 Retrieval Results

TODO: Insert and discuss retrieval comparison results from `retrieval_modes_expanded_report.md`.

Table 5.2: Retrieval modes comparison table.

Mode	R@5	R@10	P@5	P@10	MRR	Coverage	Forbidden
Vector-only	0.012	0.012	0.026	0.020	0.022	0.012	0.000
Hybrid	0.860	0.920	0.571	0.380	0.833	0.920	0.000
Graph-augmented	0.961	1.000	0.562	0.359	0.872	1.000	0.000

TODO: Overall comparison. Explain that graph-augmented retrieval improved Recall@10 from 0.920 to 1.000 compared with hybrid retrieval on the constructed benchmark, while precision decreased slightly because graph expansion adds supporting context.

Table 5.3: Category-level result table.

Category	Q	Vec R@10	Hyb R@10	Graph R@10	Hyb Cov.	Graph Cov.
Calculation/table lookup	5	0.067	1.000	1.000	1.000	1.000
Comparison QA	4	0.000	1.000	1.000	1.000	1.000
Definition QA	7	0.000	1.000	1.000	1.000	1.000
Direct QA	19	0.000	1.000	1.000	1.000	1.000
Document guidance QA	10	0.000	0.683	1.000	0.683	1.000
Exception-based QA	11	0.045	0.955	1.000	0.955	1.000
Multi-hop QA	2	0.000	0.583	1.000	0.583	1.000
Procedure QA	7	0.000	0.857	1.000	0.857	1.000
Scenario-based QA	4	0.000	1.000	1.000	1.000	1.000

TODO: Graph-required query results. Report graph-required query results: 25 graph-required queries achieved 1.000 retrieval pass, answer pass, citation pass, end-to-end pass, and average quality score 100.00 in the final graph-augmented end-to-end run.

5.5 End-to-End Results

TODO: Report only the final generated metrics. Do not generalize the pass rate beyond the benchmark scope.

Table 5.4: End-to-end evaluation summary table.

Metric	Value
Benchmark queries	69
End-to-end pass rate	1.000
Retrieval pass rate	1.000
Answer pass rate	1.000
Citation validation pass rate	1.000
Quality validation pass rate	1.000
Low-information quotes	0
Unsupported article numbers	None
Unretrieved citations	None

TODO: Overall pass rate. Explain the 1.000 end-to-end pass rate on the constructed benchmark without extending the claim beyond benchmark scope.

TODO: Category-level results. Add category-level end-to-end pass rates from `final_thesis_evaluation_report.md`. The final report states that all evaluated topics reached 1.000 retrieval, answer, citation, and end-to-end pass rates in the graph-augmented run.

TODO: Difficulty-level results. Report difficulty-level results: easy 15 queries, medium 32 queries, hard 22 queries; each group reached 1.000 retrieval pass, answer pass, citation pass, and end-to-end pass in the final graph-augmented run.

TODO: Normative-hierarchy-required results. Report normative-hierarchy-required results: 22 true and 47 false queries; both groups reached 1.000 end-to-end pass rate in the final graph-augmented run.

5.6 Case Studies

TODO: Add concise case studies using actual retrieved contexts and generated answers from the final evaluation artifacts.

TODO: Minor worker question. Show how the system retrieves provisions for workers under 15 years old and cites the relevant Labor Code and guidance documents.

TODO: Retirement age table lookup. Show how appendix or table chunks are retrieved and cited for retirement-age lookup questions.

TODO: Labor contract content with circular guidance. Show how graph expansion connects Labor Code contract provisions to circular guidance.

TODO: Dismissal dispute and court procedure. Show how labor discipline or dismissal content is connected with labor-dispute procedure provisions.

5.7 Error Analysis and Limitations

The 100% result is measured on the constructed 69-query benchmark and should not be interpreted as universal legal correctness.

TODO: Discuss remaining limitations: scoped corpus, constructed benchmark, deterministic validation rather than expert review, extractive answer style, dependence on chunking and reference parsing quality, and external legal updates outside the indexed corpus. Move verbose failure-taxonomy or raw examples to Appendix G if needed.

Chapter 6

Discussion

6.1 Key Findings

TODO: Discuss findings supported by the generated evaluation artifacts: graph augmentation improved required citation retrieval over hybrid retrieval on the constructed benchmark, and the complete graph-augmented extractive pipeline passed deterministic validation checks.

6.2 Impact of Hierarchical Chunking

TODO: Explain how article, clause, point, and appendix-aware chunking affected evidence localization, citation formatting, retrieval filters, and validation.

6.3 Benefits of Graph-Augmented Retrieval

TODO: Discuss observed benefits for document-guidance, multi-hop, procedure, exception-based, and table-lookup queries. Use the category-level table from Chapter 5 as evidence.

6.4 Role of Normative Hierarchy in Legal QA

TODO: Explain the role of codes, decrees, circulars, and appendices in context ordering and retrieval expansion. Avoid claiming that rank ordering alone resolves legal conflicts.

6.5 Citation Grounding and Hallucination Control

TODO: Discuss how extractive generation and rule-based citation validation reduce unsupported article numbers and unretrieved citations within the benchmark scope.

6.6 Practical Applications

TODO: Discuss responsible uses for legal information access, HR support, study, and research. State that real legal decisions should be verified by qualified professionals.

6.7 Limitations

The 100% result is measured on the constructed 69-query benchmark and should not be interpreted as universal legal correctness.

Table 6.1: Limitations table.

Limitation	Thesis Treatment
Scoped corpus	State that the system covers six indexed legal documents and does not cover all Vietnamese law.
Constructed benchmark	Interpret 1.000 scores as benchmark-specific evidence only.
Deterministic validation	Clarify that answer quality was not reviewed by human legal experts or LLM-as-Judge.
Extractive answers	Explain that extractive answers improve grounding but may be less fluent or adaptive.
Upstream parsing dependence	Discuss how chunking, metadata, and reference extraction errors can affect retrieval.
Legal updates	Note that newer or external legal sources require corpus refresh and re-evaluation.

TODO: Expand this limitations table into prose after the final thesis argument is drafted.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

TODO: Conclude with evidence-backed claims only: the implemented system integrates hierarchy-aware chunking, Qdrant, Neo4j, graph-augmented retrieval, extractive answer generation, and deterministic citation validation for a scoped Vietnamese labor-law corpus.

7.2 Future Work

TODO: Introduce concrete next steps that address the current benchmark and validation limitations.

- **Expert-validated benchmark.**

TODO: Create a benchmark reviewed by Vietnamese labor-law experts and compare it with the current constructed benchmark.

- **LLM-as-Judge evaluation.**

TODO: Add LLM-as-Judge evaluation only as a future complementary method with strict grounding checks, not as the current final evaluation method.

- **Human legal review.**

TODO: Evaluate answer usefulness, legal nuance, and risk with qualified legal reviewers.

- **Larger legal corpus.**

TODO: Expand the corpus to additional labor-related laws, implementing documents, official guidance, and newer amendments.

- **Improved generative answering.**

TODO: Explore constrained abstractive generation while preserving citation grounding, refusal behavior, and article-number validation.

- **Real-world deployment.**

TODO: Study production requirements such as legal update monitoring, user feedback, audit logs, privacy, latency, and responsible-use warnings.

References

- [1] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. 2020.
- [2] TODO. *TODO: Verified official sources for Vietnamese labor-law legal documents*. Placeholder citation key. Replace with official legal-document citation metadata before final submission.
- [3] TODO. *TODO: Verified reference for GraphRAG or graph-augmented retrieval*. Placeholder citation key. Replace with verified bibliographic metadata before final submission.
- [4] TODO. *TODO: Verified reference for legal question answering*. Placeholder citation key. Replace with verified bibliographic metadata before final submission.
- [5] TODO. *TODO: Verified reference for Qdrant vector database*. Placeholder citation key. Replace with verified bibliographic metadata before final submission.
- [6] TODO. *TODO: Verified reference for semantic reranking or cross-encoder reranking*. Placeholder citation key. Replace with verified bibliographic metadata before final submission.
- [7] TODO. *TODO: Verified reference for Sentence Transformers*. Placeholder citation key. Replace with verified bibliographic metadata before final submission.

Appendix A

List of Legal Documents

TODO: List the six legal documents used in the final corpus, including official titles, document IDs, effective dates, source URLs, document type, normative rank, and chunk counts.

Table A.1: Appendix document list placeholder.

Document ID	Official Title	Source URL	Notes
45-2019-qh14	TODO	TODO	Labor Code 2019
92-2015-qh13-labor-only	TODO	TODO	Labor-related procedure provisions
ngghi-dinh-135-2020-nd-cp	TODO	TODO	Retirement age guidance
ngghi-dinh-145-2020-nd-cp	TODO	TODO	Labor Code implementation
thong-tu-09-2020-tt-bldtbxh	TODO	TODO	Circular guidance
thong-tu-10-2020-tt-bldtbxh	TODO	TODO	Circular guidance

Appendix B

Chunk Schema

TODO: Document the JSONL schema for legal chunks and enriched chunks, including required fields, optional fields, and validation rules.

Appendix C

Graph Schema

TODO: Document Neo4j labels, relationship types, properties, constraints, indexes, and example Cypher queries.

Appendix D

Benchmark Samples

TODO: Add representative benchmark samples with question text, category, difficulty, required citations, forbidden citations, graph-required label, and normative-hierarchy-required label. Do not include fabricated examples.

Appendix E

Evaluation Metrics

TODO: Define retrieval metrics, answer validation checks, citation validation checks, and pass/fail aggregation. Include formulas used by the evaluation scripts.

Appendix F

Reproducibility Commands

TODO: Copy final reproducibility commands from `artifacts/evaluation/reproducibility_notes.md`, including retrieval-mode evaluation, end-to-end evaluation, optional vector index rebuild, and optional Neo4j graph rebuild.

Appendix G

Example Retrieved Contexts and Answers

TODO: Insert selected retrieved contexts and generated answers for the Chapter 5 case studies after choosing final examples from the evaluation outputs.